

Object-Oriented Programming in Java

Lecture 5 - Inheritance

Emily Lucia Antosch

HAW Hamburg

19.10.2025

Contents

1. Introduction	2
2. Inheritance	6
3. Object Instantiation	36
4. Referencing via Base Class	47
5. Overriding	53
6. Polymorphismus	68
7. Comparing Objects	73
8. License Notice	81

1. Introduction

1.1 Where Are We Currently?

- The last lecture was about class libraries
- You can now
 - ▶ use simple class libraries to manipulate strings or arrays
 - ▶ read all items from an array via a foreach loop
 - ▶ perform type conversions using wrapper classes,
 - ▶ perform simple mathematical calculations using the Math class.
- Today we continue with **Inheritance**.

1.1 Where Are We Currently?

1. Introduction

1. Imperative Concepts
2. Classes and Objects
3. Class Library
4. **Inheritance**
5. Interfaces
6. Graphical User Interfaces
7. Exception Handling
8. Input and Output
9. Multithreading (Parallel Computing)

1.2 The Goal of This Chapter

- You create new data types by extending existing classes with additional properties, for example to avoid duplicated source code.
- You use visibility modifiers to protect the attributes of a class from direct external access.

2. Inheritance

2.1 Inheritance

- Class takes over (“inherits”) variables and methods of an existing class
- Goal: Reuse of existing classes
- Example and UML notation:
 - ▶ Class A exists
 - ▶ Class B is created and inherits from A
- Terms:
 - ▶ Class A: Superclass (base class, parent class)
 - ▶ Class B: Subclass (derived class, child class)
 - ▶ Inheritance: Derivation, English: inheritance

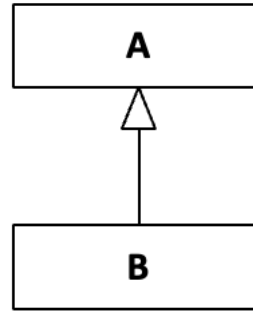


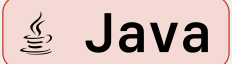
Figure 1: Simple example of inheritance

2.1 Inheritance

2. Inheritance

- Derivation from base class using extends:

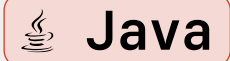
```
1 class ClassName extends BaseClass {  
2     Attributes  
3     Methods  
4 }
```





Example

```
1  class A {  
2      // ...  
3  }  
4  
5  class B extends A {  
6      // ...  
7  }
```



Task 1

- Create the following classes:
 - ▶ Person: Objects contain the name
 - ▶ Pilot: Objects contain the name and previous flight hours
 - ▶ Executable class that creates a Pilot object and outputs the name

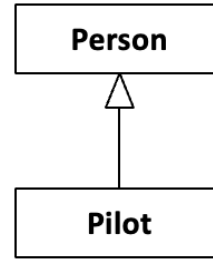
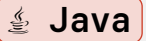


Figure 2: Pilot inherits from Person

2.1 Inheritance

2. Inheritance

```
1  public class Person {
2      String name;
3  }
4
5  public class Pilot extends Person {
6      int flightHours;
7  }
8
9  public class PilotDemo {
10     public static void main(String[] args) {
11         Pilot pilot = new Pilot();
12
13         pilot.name = "Lukas Luft";
14         pilot.flightHours = 1482;
15         System.out.println("Name: " + pilot.name);
16     }
17 }
```



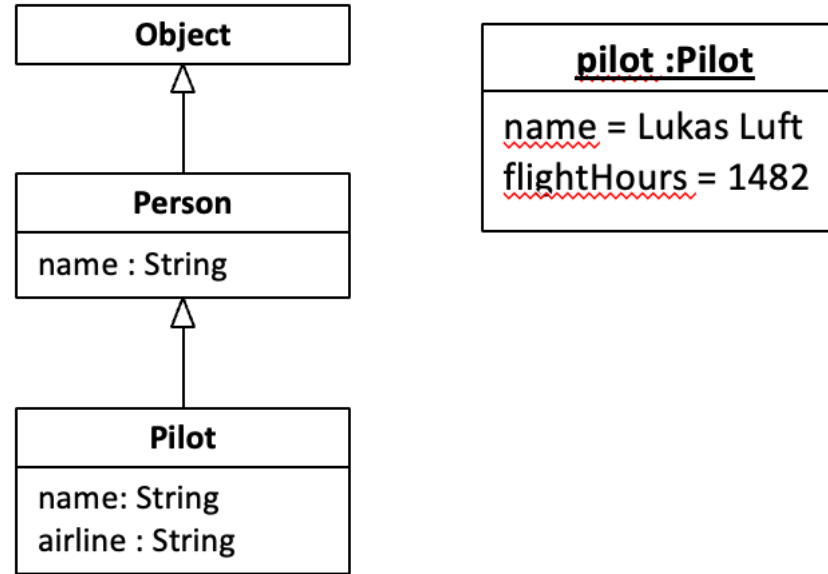
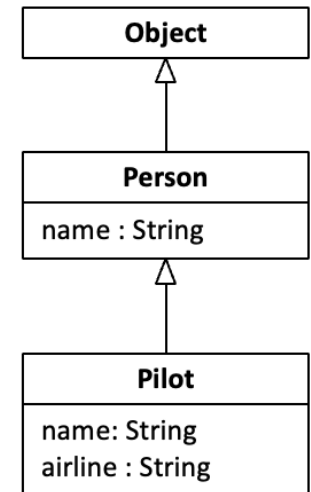
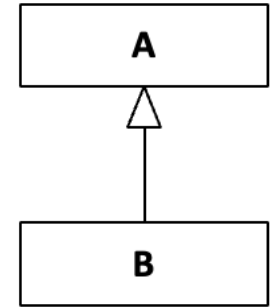


Figure 3: Inheritance of attributes

2.1 Inheritance

- Class B can add new variables and methods
- Terms:
 - ▶ Specialization: Class B is more specialized than Class A
 - ▶ Generalization: Class A is more general than Class B
- Example:
 - ▶ Class Pilot has inherited from Person and added flightHours
 - ▶ A Pilot is a Person, i.e. Person is more general than Pilot.



2.1 Inheritance

2. Inheritance

```
1  public class Person {  
2      String name;  
3  }  
4  
5  public class Pilot extends Person {  
6      int flightHours;  
7  }
```



2.1 Inheritance

- Data encapsulation (information hiding): Variables protected from external access
- Restrictions on access to classes, variables and methods through modifiers
- Mental model: “Visibility” (i.e. is element visible or known?)

Modifier	UML	Visibility	Class	Attributes	Methods
public	+	All Classes	X	X	X
protected	#	Subclasses, classes of the package		X	X
private	-	Within the class		X	X
<none>	~	Classes of the package	X	X	X

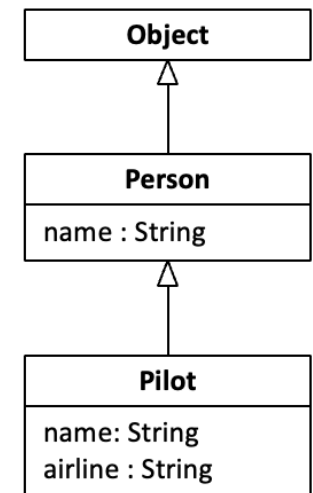
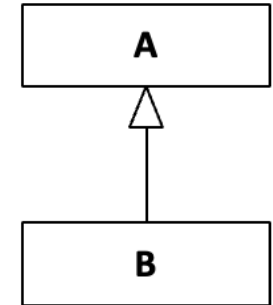
Table 1: Formats and Flags

? Question

Is `private` allowed as a modifier for constructors?

2.1 Inheritance

- Class B inherits all visible variables and methods of Class A
- Class B possesses variables and methods from A and can use them (as if they were defined in Class B)
- Example:
 - ▶ Object of Class Pilot uses variable name from base class Person



2.1 Inheritance

2. Inheritance

```
1  public class PilotDemo {
2      public static void main(String[] args) {
3          Pilot pilot = new Pilot();
4
5          pilot.name = "Lukas Luft";
6          pilot.flightHours = 1482;
7          System.out.println("Name: " + pilot.name);
8      }
9  }
```



? Question

- What do you think, which components of a class are not inherited?
- Not passed to derived class:
 - ▶ Constructors and destructors
 - ▶ Class variables and class methods (modifier static)
 - ▶ Private variables and methods (modifier private)

- Notes:
 - ▶ Static elements never inherited, as they are bound to a class and not to a concrete object
 - ▶ Private elements are present in subclass, but it cannot directly access them

2.1 Inheritance

- Subclasses can be further inherited.
- Any number of subclasses can be derived from one class.
- However, inheriting from multiple base classes is not possible (multiple inheritance)

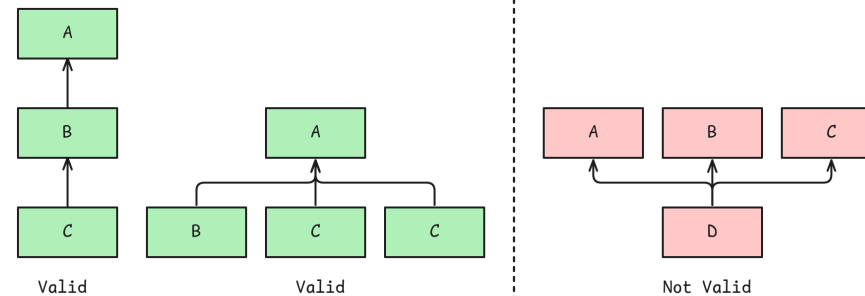
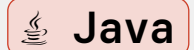


Figure 6: Possible structures for inheritance

? Question

- What do you think, which base class does Person have?

```
1  public class Person {  
2      String name;  
3  }
```



- You couldn't have known this before:
 - ▶ In Java a class Object is defined.
 - ▶ No base class specified. Implicitly derived from Object (extends Object)

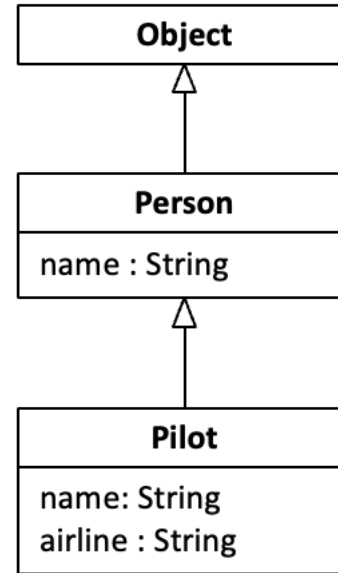


Figure 7: Object as superclass of Person

2.1 Inheritance

- Important consequence:
 - ▶ Object is the base class of every inheritance hierarchy

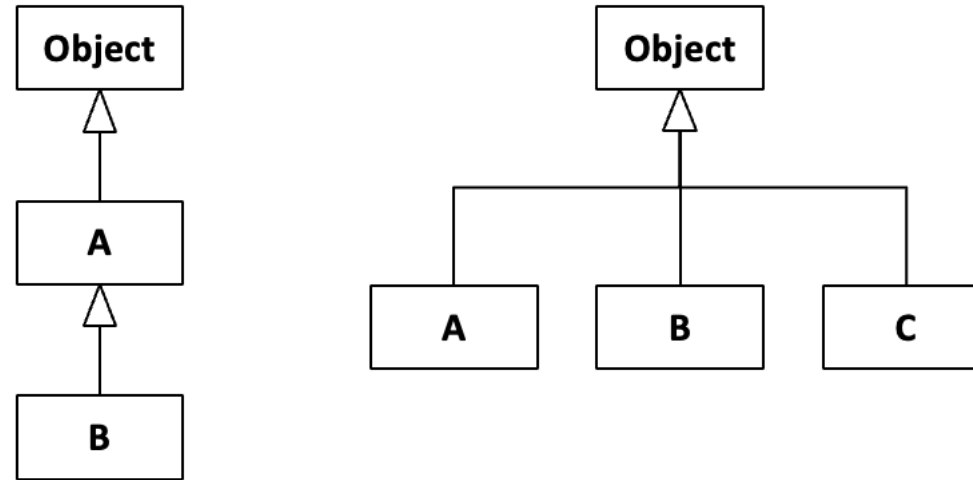


Figure 8: Object as part of every inheritance

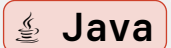
? Question

- What do you think?
 - ▶ How many classes have no base class?
 - ▶ How many classes have more than one direct base class?

2.1 Inheritance

- Important consequence:
- Every class inherits the methods defined in Object (e.g. toString())
- Example:

```
1  public class Person {  
2      String name;  
3  }  
4  
5  public class ObjectDemo {  
6      public static void main(String[] args) {  
7          Person person = new Person();  
8  
9          person.name = "Lukas Luft";  
10         System.out.println(person.toString());  
11     }  
12 }
```



Task 2

- Implement classes for geometric objects Circle, Rectangle and Square.
- Use only public variables for now.
- Don't implement any methods for now.

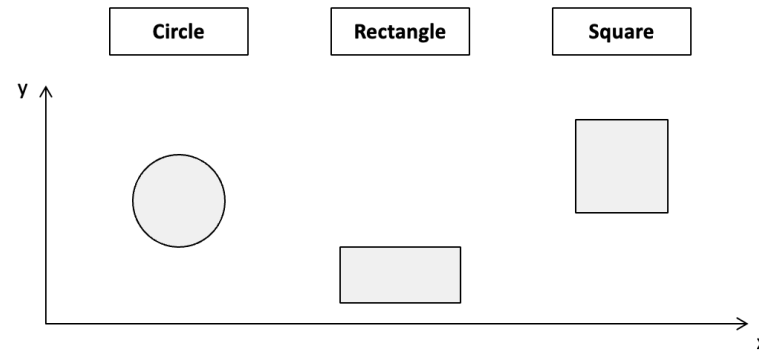
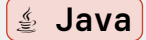


Figure 9: Geometric shapes as objects

2.1 Inheritance

2. Inheritance

```
1  public class Circle {
2      public double x, y;
3      public double radius;
4  }
5
6  public class Rectangle {
7      public double x, y;
8      public double width, height;
9  }
10
11 public class Square {
12     public double x, y;
13     public double width;
14 }
```

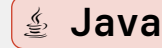


☰ Task 3

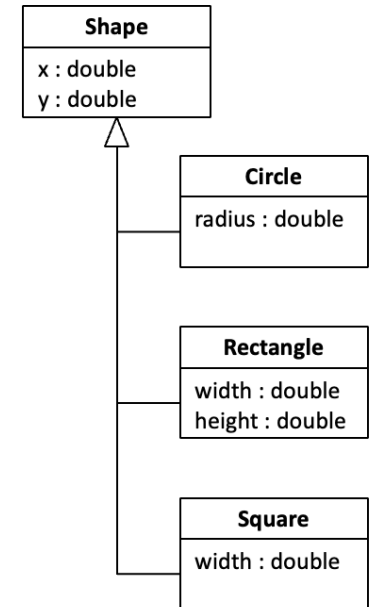
Now create a common base class!

2.1 Inheritance

```
1  public class Shape {
2      public double x, y;
3  }
4
5  public class Circle extends Shape {
6      public double radius;
7  }
8
9  public class Rectangle extends Shape {
10     public double width, height;
11 }
12
13 public class Square extends Shape {
14     public double width;
15 }
```



2. Inheritance



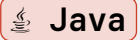
? Question

- Which variables are declared in the respective classes?

☰ Task 4

- Add a constructor for the Circle class!

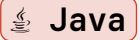
```
1  public class Shape {
2      public double x, y;
3  }
4
5  public class Circle extends Shape {
6      public double radius;
7
8      public Circle(double x, double y, double radius) {
9          this.x = x;
10         this.y = y;
11         this.radius = radius;
12     }
13 }
```



! Memorize

- Note: Variables x and y of the base class are used like “own” variables
- Hide the variables of the Shape class through the private modifier.

```
1  public class Shape {
2      private double x, y;
3  }
4
5  public class Circle extends Shape {
6      public double radius;
7
8      public Circle(double x, double y, double radius) {
9          this.x = x;
10         this.y = y;
11         this.radius = radius;
12     }
13 }
```



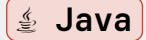
✗ Error

- The variables x and y of the base class are not visible in Circle.
- Error: In the constructor of class Circle, x and y are unknown.

2.1 Inheritance

2. Inheritance

```
1  public class Shape {
2      private double x, y;
3
4      public void setX(double x) {
5          this.x = x;
6      }
7      // Additionally getter and corresponding methods for y ...
8  }
9
10 public class Circle extends Shape {
11     public double radius;
12
13     public Circle(double x, double y, double radius) {
14         setX(x);
15         setY(y);
16         this.radius = radius;
17     }
18 }
```



3. Object Instantiation

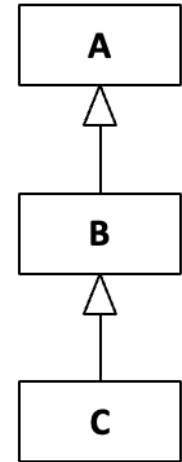
3.1 Object Creation

3. Object Instantiation

- Class C contains its own methods as well as methods from classes A and B.
- Class C contains its own variables as well as variables from classes A and B.

? Question

- What do you think?
 - ▶ How are methods of a new object of class C created?
 - ▶ How are variables of an object of class C created and initialized?
- Methods:
 - ▶ Are not created anew for each object, but are defined for the class
- Variables:
 - ▶ Start at the base class of the inheritance hierarchy
 - ▶ In each step, create and initialize variables of the corresponding (base) class
 - ▶ Initialization via constructor of the respective (base) class



3.1 Object Creation

3. Object Instantiation

- Variables for objects of class C:
 - ▶ Object contains the variables declared in class C
 - ▶ Additionally contains variables inherited from class B
 - ▶ These contain the variables inherited from class A

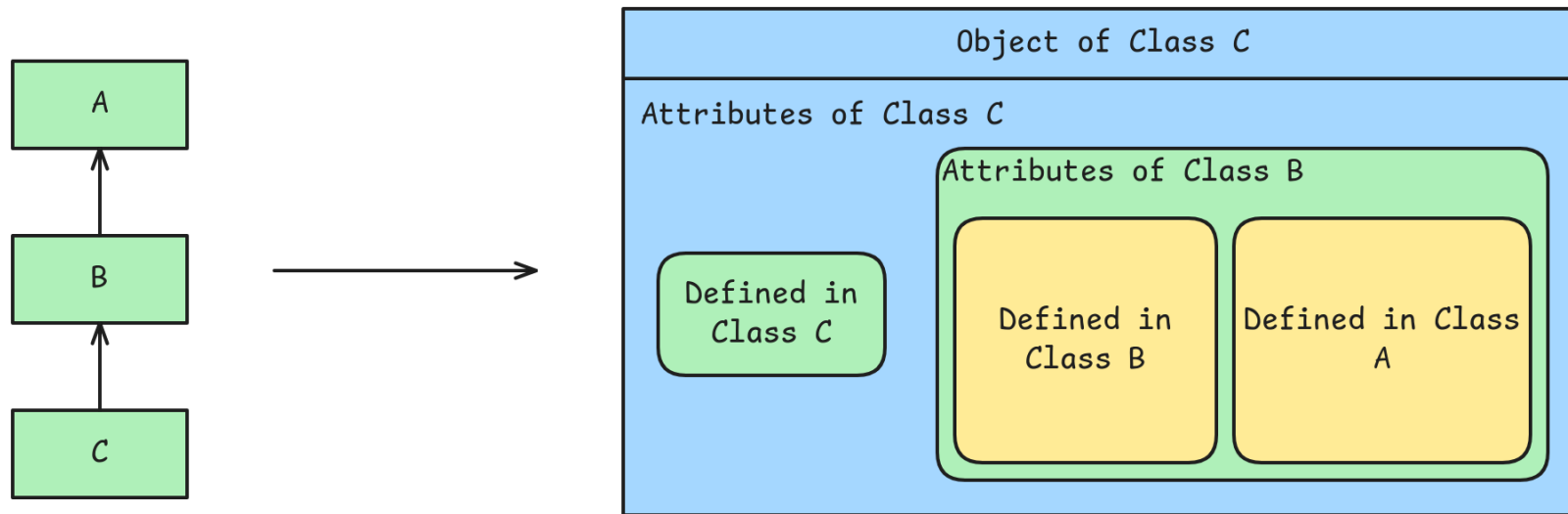


Figure 12: Composition of the object of class C

- New object of class C:
 - ▶ Traverse inheritance hierarchy upward:
 - ▶ Class C has base class B: Call to create variables from B
 - ▶ Class B has base class A: Call to create variables from A
- Create and initialize variables “from inside out” (constructor chaining):
 - ▶ Create variables from A and initialize via constructor A()
 - ▶ Create variables from B and initialize via constructor B()
 - ▶ Create variables from C and initialize via constructor C()

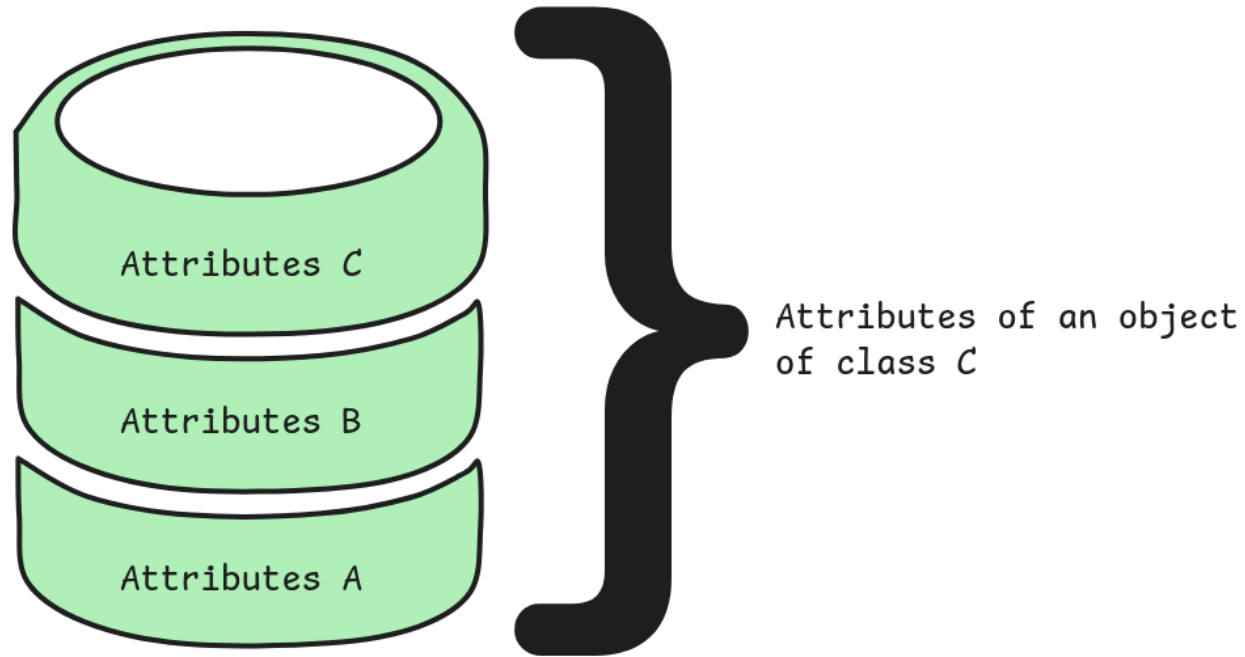


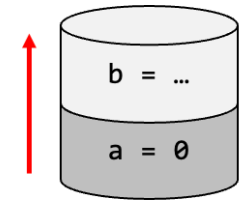
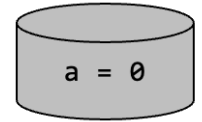
Figure 13: Variables of class C in memory

3.1 Object Creation

3. Object Instantiation

- Constructor of the base class:
 - ▶ Call via `super()` as first statement in constructor of derived class
 - ▶ If `super(...)` is missing, the default constructor of the base class is called.

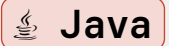
```
1  public class A {  
2      double a;  
3      // Default constructor is automatically generated  
4  }  
5  
6  public class B extends A {  
7      double b;  
8  
9      public B(double b) {  
10         super();    // Call default constructor of class A  
11         this.b = b;  
12     }  
13 }
```



3.1 Object Creation

3. Object Instantiation

- Do you remember?



Java

```
1  public class Shape {
2      private double x, y;
3  }
4
5  public class Circle extends Shape {
6      public double radius;
7
8      public Circle(double x, double y, double radius) {
9          this.x = x;
10         this.y = y;
11         this.radius = radius;
12     }
13 }
```

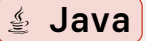
Task 5

- ▶ The variables `x` and `y` are unknown in `Circle`.
- ▶ Solve the problem by adding a constructor for the base class `Shape`.

3.1 Object Creation

3. Object Instantiation

```
1  public class Shape {
2      private double x, y;
3
4      public Shape(double x, double y) {
5          this.x = x;
6          this.y = y;
7      }
8  }
9
10 public class Circle extends Shape {
11     public double radius;
12
13     public Circle(double x, double y, double radius) {
14         super(x, y); // Matching signature to the constructor of the base class!
15         this.radius = radius;
16     }
17 }
```



Java

☰ Task 6

- Protect all attributes through the private modifier.
- Create appropriate getters and setters if necessary.

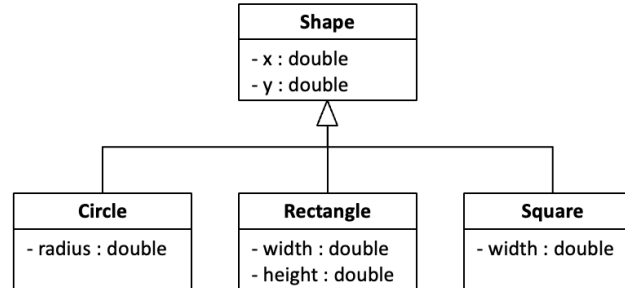
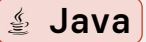


Figure 15: Structure of inheritance

3.1 Object Creation

3. Object Instantiation

```
1  public class Circle extends Shape {
2      private double radius;
3
4      public Circle(double x, double y, double radius) {
5          super(x, y);
6          this.radius = radius;
7      }
8
9      public double getRadius() {
10         return radius;
11     }
12
13     public void setRadius(double radius) {
14         this.radius = radius;
15     }
16 }
```



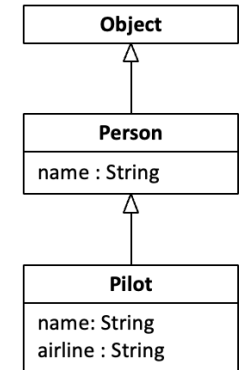
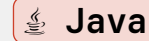
4. Referencing via Base Class

4.1 Base Class Reference

4. Referencing via Base Class

- Let's consider the following inheritance line:

```
1  public class Person {
2      String name;
3
4      public Person(String name) {
5          this.name = name;
6      }
7  }
8
9  public class Pilot extends Person {
10     String airline;
11
12     public Pilot(String name, String airline) {
13         super(name);
14         this.airline = airline;
15     }
16 }
```

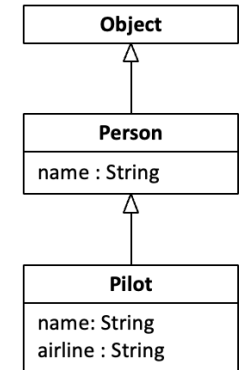


4.1 Base Class Reference

4. Referencing via Base Class

- What do you think about the following program?

```
1  public class ReferenceDemo {
2      public static void main(String[] args) {
3          Pilot pilot = new Pilot("Birgit", "Winglet Airways");
4          Person personRef = pilot;
5          Object objectRef = pilot;
6
7          System.out.println(personRef.name);
8      }
9  }
```



! Memorize

There is only one object (with data type Pilot). Object is referenced via variables with other data types than Pilot

4.1 Base Class Reference

4. Referencing via Base Class

- Class Pilot inherits from class Person and extends it
- Pilot contains Person ("Pilot is a Person") Referenceable as Person
- Object is not changed by this (i.e. object remains of type Pilot)!

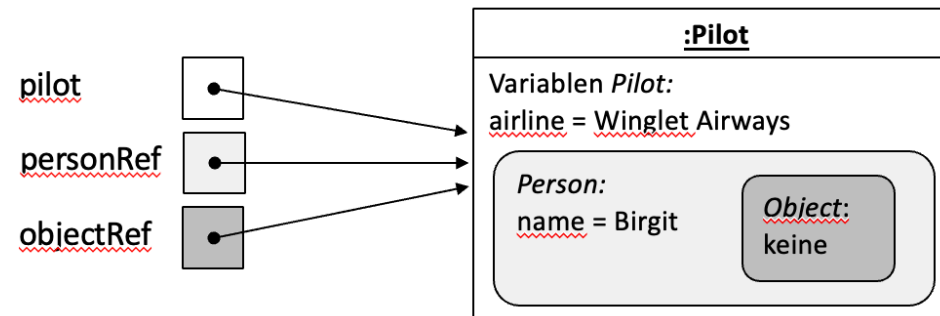


Figure 18: References to an object with base class

- In general:
 - ▶ Objects can be treated like objects of their base classes.
 - ▶ Objects referenceable via data types of their base classes
 - ▶ Reference variable can only access attributes and methods of its class

? Question

- Which accesses to attributes are allowed and which are not?

```
1  public static void main(String[] args) {  
2      Pilot pilot = new Pilot("Birgit", "Winglet Airways");  
3      Person personRef = pilot;  
4      Object objectRef = pilot;  
5  
6      System.out.println(personRef.name);  
7      System.out.println(personRef.airline);  
8      System.out.println(objectRef.name);  
9      System.out.println(objectRef.airline);  
10 }
```

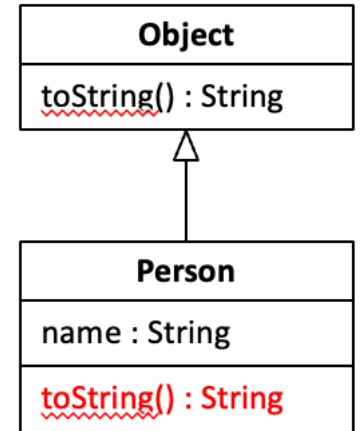


5. Overriding

5.1 Overriding Methods

- Class Person is derived from Object and therefore inherits toString() from Object and therefore defines another toString() method

```
1  public class Person {
2      private String name;
3
4      public Person(String name) {
5          this.name = name;
6      }
7
8      public String getName() {
9          return name;
10     }
11
12     public String toString() {
13         return name;
14     }
15 }
```

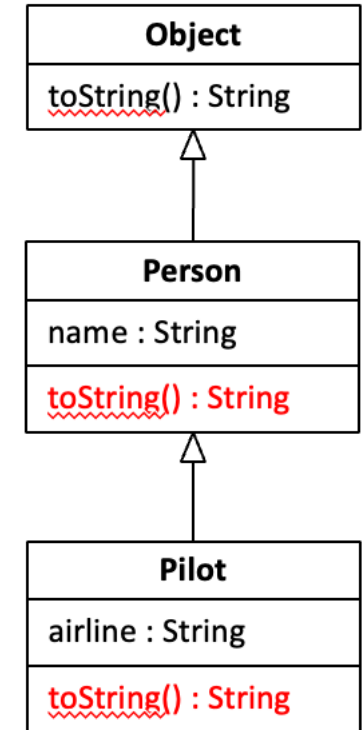


5. Overriding

5.1 Overriding Methods

- Class Pilot:
 - ▶ Derived from Person and therefore inherits toString() from Person
 - ▶ Defines another toString() method

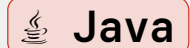
```
1  public class Pilot extends Person {  
2      private String airline;  
3  
4      public Pilot(String name, String airline) {  
5          super(name);  
6          this.airline = airline;  
7      }  
8  
9      public String toString() {  
10         return String.format("%s (%s)", getName(), airline);  
11     }  
12 }
```



? Question

What will be output?

```
1 public static void main(String[] args) {  
2     Person person = new Person("Birgit Janssen");  
3     System.out.println("person: " + person);  
4     System.out.println("person.toString(): " + person.toString());  
5  
6     Pilot pilot = new Pilot("Jan Birgerson", "Winglet Airways");  
7     System.out.println("pilot.toString(): " + pilot.toString());  
8 }
```

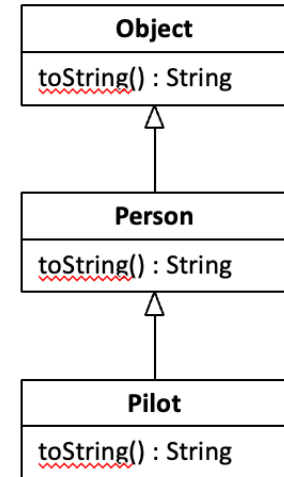


5.1 Overriding Methods

- Output:
 - ▶ `person`: Birgit Janssen
 - ▶ `person.toString()`: Birgit Janssen
 - ▶ `pilot.toString()`: Jan Birgerson (Winglet Airways)

! Memorize

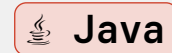
- ▶ Respective method of the corresponding class, not the superclass(es), executed
- ▶ Term: Method of the superclass overridden by newly defined methods



? Question

- What will be output?

```
1 public static void main(String[] args) {  
2     Pilot pilot = new Pilot("Jan Birgerson", "Winglet Airways");  
3     Object objectRef = pilot;  
4     Person personRef = pilot;  
5  
6     System.out.println("objectRef: " + objectRef.toString());  
7     System.out.println("personRef: " + personRef.toString());  
8 }
```



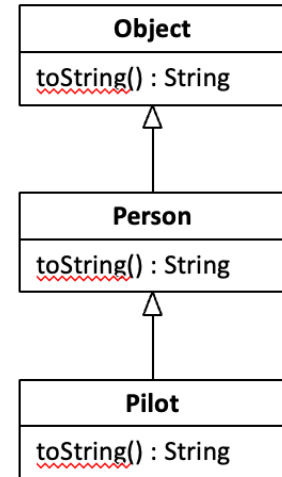
5.1 Overriding Methods

5. Overriding

? Question

- What will be output?

```
1 public static void main(String[] args) {  
2     Pilot pilot = new Pilot("Jan Birgerson", "Winglet Airways");  
3     Object objectRef = pilot;  
4     Person personRef = pilot;  
5  
6     System.out.println("objectRef: " + objectRef.toString());  
7     System.out.println("personRef: " + personRef.toString());  
8 }
```



- Output:
 - ▶ objectRef: Jan Birgerson (Winglet Airways)
 - ▶ personRef: Jan Birgerson (Winglet Airways)

! Memorize

Method of the corresponding class executed, even when referenced via superclass(es)

5.1 Overriding Methods

5. Overriding

- Access to overridden methods of the base class via reference super
- Example:

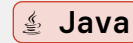
```
1  public class Pilot extends Person {
2      // Instance variable, constructor, toString() ...
3
4      public String toStringOfSuperClass() {
5          return super.toString();
6      }
7  }
8
9  public static void main(String[] args) {
10     Pilot pilot = new Pilot("Jan Birgerson", "Winglet Airways");
11     System.out.println("Pilot.toString(): " + pilot.toString());
12     System.out.println("super.toString(): " + pilot.toStringOfSuperClass());
13 }
```

 **Java**

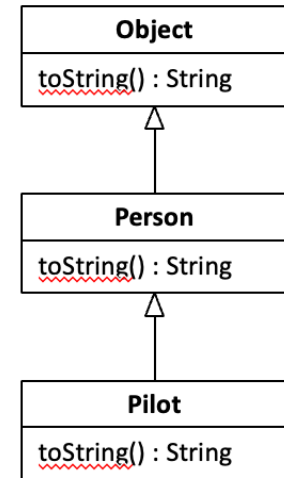
5.1 Overriding Methods

- Access to overridden methods of the base class via reference super
- Example:

```
1  public class Pilot extends Person {
2      // Instance variable, constructor, toString() ...
3
4      public String toStringOfSuperClass() {
5          return super.toString();
6      }
7  }
8
9  public static void main(String[] args) {
10     Pilot pilot = new Pilot("Jan Birgerson", "Winglet Airways");
11     System.out.println("Pilot.toString(): " + pilot.toString());
12     System.out.println("super.toString(): " + pilot.toStringOfSuperClass());
13 }
```



5. Overriding



- Output:
 - ▶ Pilot.toString(): Jan Birgerson (Winglet Airways)
 - ▶ super.toString(): Jan Birgerson

- Variables with modifier final are constants.
 - ▶ Value cannot be changed after first assignment

? Question

- What do you think?
 - ▶ What does final do for classes?
 - ▶ What does final do for methods?

•

•

5.1 Overriding Methods

- Variables with modifier final are constants.
 - Value cannot be changed after first assignment

? Question

- What do you think?
 - What does final do for classes?
 - What does final do for methods?
- Classes:
 - Class with modifier final cannot be derived
 - Example: String class
- Methods:
 - Method with modifier final cannot be overridden in subclass

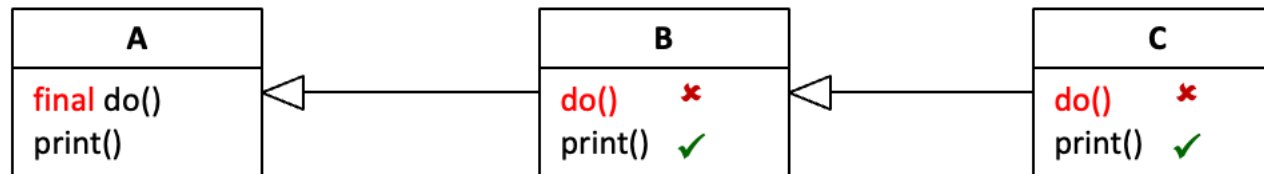
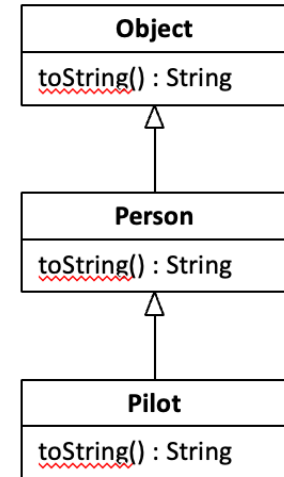


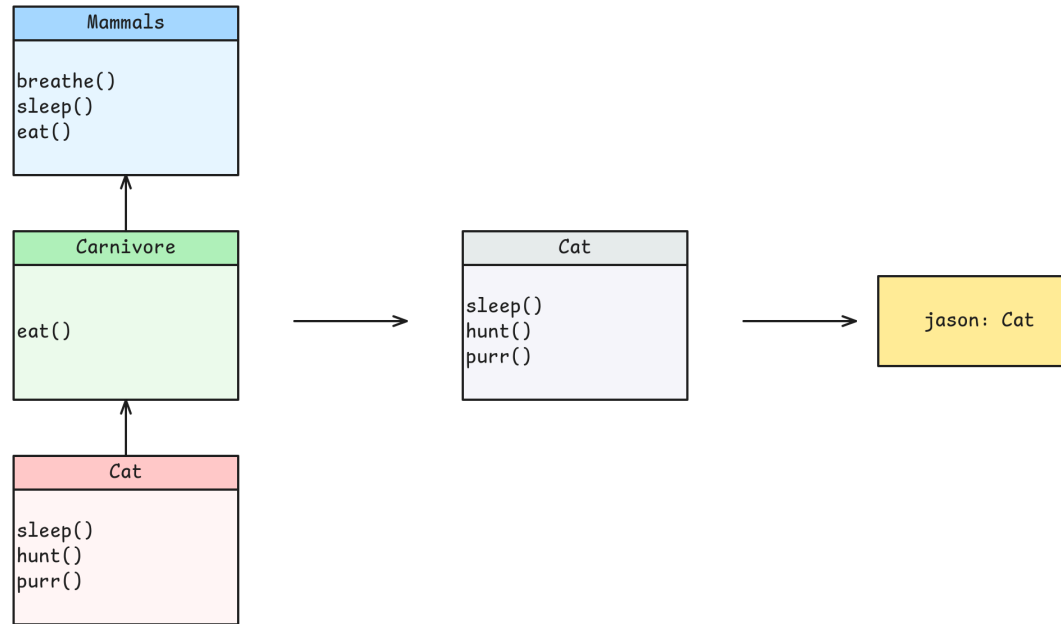
Figure 2: final keyword in inheritance

5.1 Overriding Methods

- Overriding methods:
 - ▶ Methods inherited from the base class may be redefined.
 - ▶ Terms: Overriding (or also overwriting)
 - ▶ When called, the overriding method (“newest version”) is executed
 - ▶ Call the hidden method `name()` of the base class via `super.name()`
 - ▶ Modifier `final` prevents overriding in subclasses
- Overriding attributes:
 - ▶ Derived class can override variables of the base class in the same way

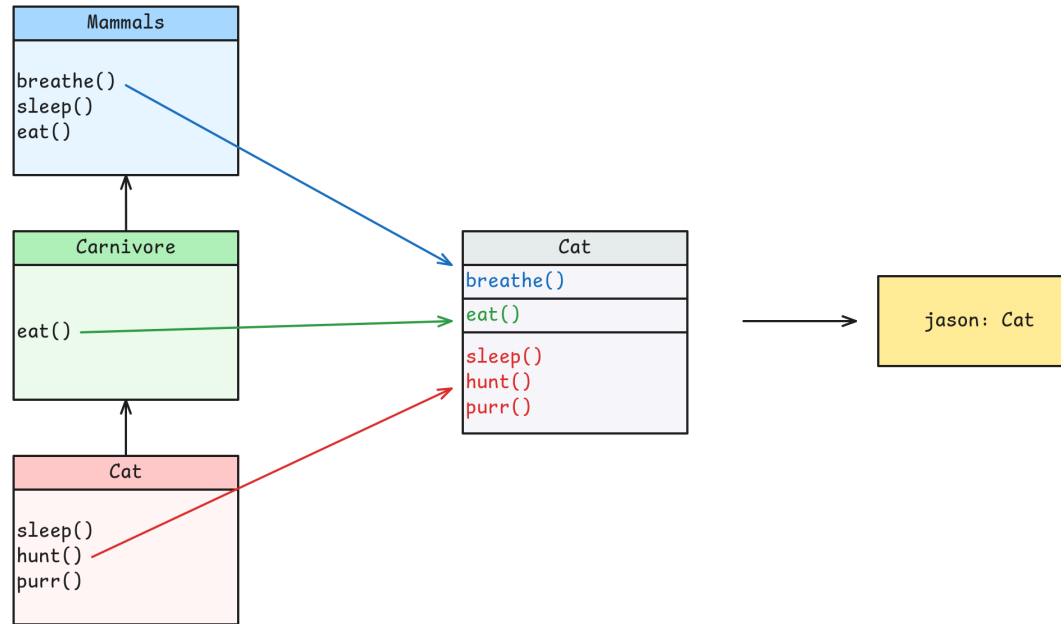
? Question

- Which methods does Jason, the cat, have?
- From which classes does each method definition come?



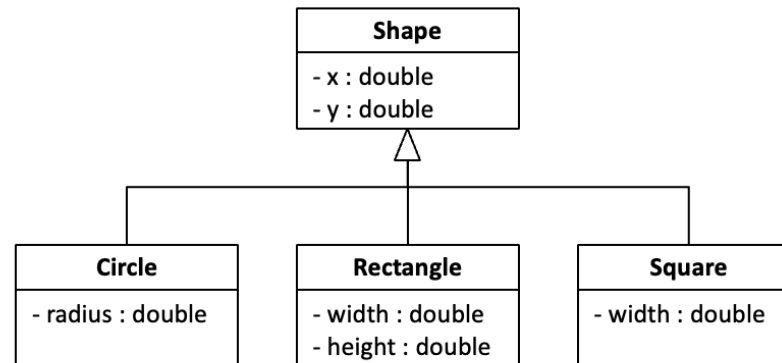
? Question

- Which methods does Jason, the cat, have?
- From which classes does each method definition come?



Task 7

- Add methods `getArea()` to determine the area of an object.
- Create the following executable program:
 - ▶ Stores one object each `Circle`, `Rectangle` and `Square` in a common list
 - ▶ Determines sum of areas from this list

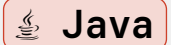


5.1 Overriding Methods

5. Overriding

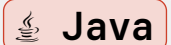
Klasse Circle:

```
1  public double getArea() {  
2      return Math.PI * radius * radius;  
3  }
```



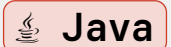
Klasse Rectangle:

```
1  public double getArea() {  
2      return width * height;  
3  }
```



Klasse Square:

```
1  public double getArea() {  
2      return width * width;  
3  }
```

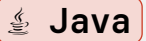


5.1 Overriding Methods

5. Overriding

- Executable program:

```
1  public static void main(String[] args) {  
2      ArrayList<Shape> shapes = new ArrayList<Shape>();  
3      shapes.add(new Circle(2.0, 3.0, 1.0));  
4      shapes.add(new Rectangle(-1.0, 0.0, 3.5, 4.0));  
5      shapes.add(new Square(0.0, 0.0, 2.5));  
6  
7      double sumArea = 0.0;  
8      for (Shape shape : shapes) {  
9          sumArea += shape.getArea();  
10     }  
11  
12     System.out.println("Overall area of shapes = " + sumArea);  
13 }
```



- Method `getArea()` must also be implemented in `Shape`
- However, it is not used, but overridden by subclasses: Very ugly!
- We will learn a more elegant solution later.

6. Polymorphismus

- The good news:
 - ▶ Only a new term, otherwise everything is already known
 - ▶ No, really. Honestly. Really true ...
- Polymorphism:
 - ▶ Literal meaning: “Many forms”
 - ▶ Methods with the same name can take multiple forms.
 - ▶ In other words: Multiple implementations of methods with the same name
 - ▶ Typical characteristic of object-oriented languages

? Question

- Where have we already encountered this?

? Question

- Where have we already encountered this?
- Methods with the same name in the same class: Overloading
- Methods with the same name in inheritance line: Overriding (also: Overwriting)

6.1 Polymorphism

- Overloading:
 - ▶ Methods in class have the same name
 - ▶ Must have different signatures (i.e. different parameter types)
- Overriding / Overwriting:
 - ▶ Methods in inheritance line have the same name
 - ▶ Must have the same signature (i.e. same name and parameter types)

MathFunctions
<code>max(int, int) : int</code> <code>max(int, int, int) : int</code> <code>max(double, double): double</code> <code>max(double[]): double</code>

Figure 6: Class MathFunctions

6.1 Polymorphism

6. Polymorphism

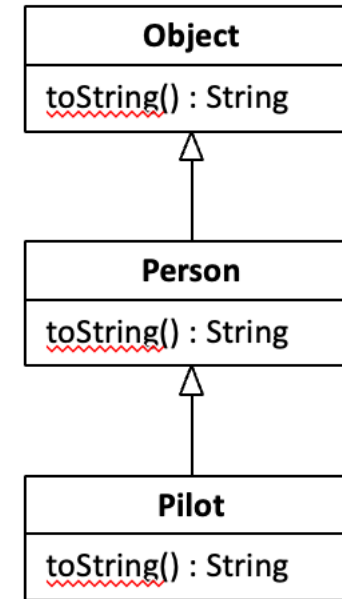


Figure 7: Inheritance of `toString()`

7. Comparing Objects

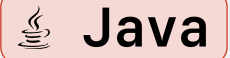
7.1 Comparison Operator (==)

7. Comparing Objects

? Question

- Class Point contains the variables x and y
- What will be output?

```
1  Point a = new Point(1, 2);  
2  Point b = new Point(7, 3);  
3  System.out.println(a == b);
```



? Question

- And now?

```
1  Point a = new Point(1, 2);  
2  Point b = new Point(1, 2);  
3  System.out.println(a == b);
```

**Java**

? Question

- Und nun?

```
1  Point a = new Point(1, 2);  
2  Point b = a;  
3  System.out.println(a == b);
```



7.1 Comparison Operator (==)

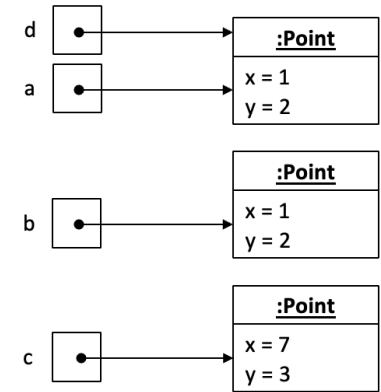
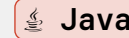
7. Comparing Objects

- Comparison operator compares whether variables have the same content
 - ▶ Content is respectively a reference to an object
 - ▶ Comparison only true when variables reference the same object



Example

```
1 Point a = new Point(1, 2);
2 Point b = new Point(1, 2);
3 Point c = new Point(7, 3);
4 Point d = a;
```



✗ Error

- `a == b`: Different objects (with same values)
- `a == c`: Different objects (and values)

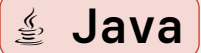
✓ Success

- `a == d`: Same object: same reference

7.2 equals()-Methode

7. Comparing Objects

```
1  public boolean equals(Object obj) {  
2      // Method body  
3      // Return a value of type boolean  
4  }
```



- Comparison whether all variables of two referenced objects have the same values
- Method is already defined in class Object
- Overriding in own classes:
 - ▶ Class Object cannot know which variables you add in subclasses
 - ▶ Therefore override method if necessary to compare added attributes
 - ▶ In IntelliJ IDEA this is conveniently possible via the Generate menu.

? Question

- Given class Point with x and y coordinates
- What result do the comparisons in the table provide?

Quelltext	a == b	<u>a.equals(b)</u>
Point a = new Point(10, 25); Point b = new Point(10, 2);	?	
Point a = new Point(10, 25); Point b = new Point(10, 25);	?	
Point a = new Point(10, 25); Point b = a;	?	

? Question

- Given class Point with x and y coordinates
- What result do the comparisons in the table provide?

Quelltext	a == b	<u>a.equals(b)</u>
Point a = new Point(10, 25); Point b = new Point(10, 2);	<u>false</u>	<u>false</u>
Point a = new Point(10, 25); Point b = new Point(10, 25);	<u>false</u>	<u>true</u>
Point a = new Point(10, 25); Point b = a;	<u>true</u>	<u>true</u>

8. License Notice

8.1 Attribution

- This work is shared under the CC BY-NC-SA 4.0 License and the respective Public License
- <https://creativecommons.org/licenses/by-nc-sa/4.0/>
- This work is based off of the work Prof. Dr. Marc Hensel.
- Some of the images and texts, as well as the layout were changed.
- The base material was supplied in private, therefore the link to the source cannot be shared with the audience.